GenSpark

Tic-Tac-Toe with AI Opponent

Tic-Tac-Toe

Overview

Build a Tic-Tac-Toe game that runs in the terminal. It supports two players or one player against a computer opponent. This is an individual assignment. Write the plan, code, and tests yourself. Do not use AI tools for any part of it, including chat assistants and AI code completion, so turn off AI features in your IDE before you start. You may use the course lessons and official documentation. Your repository history will be reviewed.

Required tools: Python 3.10 or later, an IDE of your choice, GitHub Desktop, and a GitHub account.

Game Requirements

1. The board is 3×3. X always goes first.
2. Players enter a move as a row and a column, numbered from 1, such as `2 3`.
3. The board is printed after every move.
4. A move to a taken cell, a cell off the board, or non-numeric input shows a message and lets the player try again. It does not use up the turn.
5. The game detects a win (across, down, or diagonal) and a draw.
6. At the start, the player chooses two players or versus computer. In versus

computer mode, the human plays X and the computer plays O, and the player picks Easy or Hard.

7. Easy picks a random empty cell. Hard uses the minimax algorithm and never loses when it plays second.

Design Requirements

- Keep the game rules separate from printing and typing. Only `main.py` uses `print()` and `input()`.
- Use an abstract base class for AI players, built with Python's `abc` module, with a `choose_move(board, mark)` method. `EasyAI` and `HardAI` are subclasses. The game calls `choose_move` without knowing which level it has.
- The Board keeps its grid in an attribute that starts with an underscore. Other code uses the Board through its methods.
- Pass a `random.Random` object into the AI classes so tests can seed it.
- Your automated grader calls the names in the table below, so use them exactly.

Class	Name	Takes	Returns
Board	`Board()`	nothing	a new empty 3×3 board
Board	`place_mark(row, col, mark)`	row and column from 0, "X" or "O"	`True` if placed, `False` if taken or off the board
Board	`remove_mark(row, col)`	row and column from 0	nothing
Board	`empty_cells()`	nothing	a list of `(row, col)` tuples
Board	`has_won(mark)`	"X" or "O"	`True` or `False`
Board	`is_full()`	nothing	`True` or `False`
AI base	`choose_move(board, mark)`	a Board, "X" or "O"	a `(row, col)` tuple

class			
EasyAI, HardAI	`EasyAI(rng)`, `HardAI(rng)`	a `random.Random` object	an AI object

Rows and columns inside your code start at 0. Convert from the player's numbers in one place only, in `main.py`.

Files

File	Purpose
`board.py`	The Board class
`ai.py`	The AI classes
`game.py`	Turns, modes, and win or draw checks
`main.py`	Menus, prompts, and printing
`tests/`	Your pytest tests
`PLAN.md`	Your written plan
`README.md`	How to run the game and the tests
`.github/workflows/ci.yml`	Your pipeline

Part 1: Setup

1. Create a public repository named `tic-tac-toe` with a README and the Python .gitignore template.
2. Add `.idea/` and `.vscode/` to `.gitignore`.

3. Create a virtual environment, install pytest, and save your packages to `requirements.txt`.
4. Create an empty `conftest.py` in the project root so your tests can find your code, and create a `tests` folder.
5. Commit `.gitignore`, `conftest.py`, and `requirements.txt` straight to main. This is the first of two direct commits to main allowed.

Part 2: Plan

Before you write code, create `PLAN.md` in the project root with these sections:

1. **Goal.** Two sentences on what the game is and who plays it.
2. **Requirements.** A table with an ID, the requirement, and how you will verify it. Write at least 6. Each must be testable. "The AI should be smart" is not. "Hard never loses on 3×3 when it plays second" is.
3. **Components.** Each class or file, with one sentence on its single responsibility.
4. **Decisions.** At least 2 decisions, each with the options you considered, what you chose, and why.

Commit `PLAN.md` straight to main. This is the second and last direct commit to main. When you finish, add a short "What changed" section at the end: what you changed from your plan, and why.

Part 3: Build

Build on two branches, each merged to main by a pull request:

- `feature/board-and-rules`: the Board, the two-player game in `main.py`, and the CI workflow
- `feature/ai-opponents`: the AI classes and versus computer mode

For each branch:

1. Create the branch from main in GitHub Desktop.
2. Commit often, with messages that start with a verb.
3. Push the branch and open a pull request into main. Write a 2 to 3 sentence description of what changed and how you tested it.
4. Wait for the checks. If one is red, read the log, fix the problem on the same branch, and push again. Merge only when the checks are green.
5. Merge on GitHub, delete the branch, then switch to main in GitHub Desktop and pull.

Make at least 8 commits in total across the two branches.

The CI workflow

Add a file named `.github/workflows/ci.yml` in the first branch. It runs your tests whenever a pull request is opened or main changes. Use this workflow, not the starter workflow offered in the Actions tab:

```
name: CI

on:
  push:
    branches: [main]
  pull_request:

jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v6
      - uses: actions/setup-python@v5
        with:
          python-version: "3.12"
      - name: Install dependencies
        run: pip install -r requirements.txt
```

```
      - name: Run tests
        run: pytest
```

When you open your first pull request, the checks section shows the test job running, then a green check or a red X.

Tests

Write at least 12 pytest tests, all passing. Cover at least:

- a new board is empty
- a taken cell is rejected and the first mark stays
- off-board positions are rejected, including negative numbers
- a win across, down, and on both diagonals
- a full board with no winner is a draw
- the abstract AI class cannot be created directly
- Easy always returns an empty cell
- Hard takes a winning move when one exists
- Hard never loses on 3×3
- `choose_move` leaves the board unchanged

Tests must never depend on unseeded randomness.

Part 4: Finish

1. Write your `README.md`: what the game is, how to run it, and how to run the tests.
2. Add the "What changed" section to `PLAN.md`.
3. Confirm both pull requests are merged, main has the finished game, and the Actions tab shows a green run on main.

Helpful Links

Topic	Link
Hello GitHub Actions (hands-on course)	GitHub Learn - Skills
Building and testing Python with Actions	GitHub Actions
pytest getting started	Get Started Pytest
Python abc module (abstract classes)	Abstract Base Class
Python random module	Random Generator

Deliverable

Submit your public repository URL as text. Keep the repository public until grading is complete.

Grading Rubric (100 points)

Criteria	Points
Rules: turns alternate, win and draw detected, invalid moves rejected without losing the turn	20
AI: Easy is random, Hard never loses on 3×3	20
Design: abstract base class, polymorphism, encapsulated Board, no print or input outside main.py	15
Tests: at least 12, all passing, seeded randomness	15
Git: two feature branches merged by pull request, at least 8 commits, no feature work	10

committed to main	
CI: ci.yml runs the tests, checks pass on the merged pull requests, and the latest run on main is green	10
Plan: PLAN.md has the four sections, at least 6 testable requirements, at least 2 decisions, and a "What changed" section	5
README: how to run the game and the tests	5
Total	**100**

Submit your assignment

Fill in the text fields and upload your files below.

Text Field(s)

Enter you GitHub Repo. (URL) *

https://example.com

Drop your files here to upload

Any file type is allowed.